

Analyse et comparaison des techniques de prétraitement pour l'apprentissage automatique

Auteur : Louraoui Ayoub
Encadrant : M. Mestari Mohammed
Groupe : 4IADM G1
Date : 2 mai 2026

*

Table des matières

1	Introduction	1
2	Présentation des jeux de données	1
2.1	Jeu de données Titanic	1
2.2	Jeu de données 20 Newsgroups	2
3	Méthodologie	2
3.1	Fondements théoriques	2
3.1.1	Standardisation	2
3.1.2	MinMaxScaler	2
3.1.3	OneHotEncoder	3
3.1.4	OrdinalEncoder	3
3.1.5	TF-IDF (Term Frequency - Inverse Document Frequency)	3
3.2	Stratégie de prétraitement - Titanic	3
3.2.1	Groupes de colonnes	3
3.2.2	Architecture du pipeline	3
3.3	Stratégie de prétraitement - Textes	4
3.3.1	Représentations vectorielles comparées	4
3.3.2	Pipelines textuels	4
3.4	Protocole expérimental	4
4	Résultats expérimentaux	4
4.1	Résultats - Titanic	4
4.1.1	Validation croisée	4
4.1.2	Optimisation des hyperparamètres	5
4.1.3	Évaluation finale sur test	5
4.2	Résultats - 20 Newsgroups	5
4.2.1	Validation croisée	5
4.2.2	Évaluation finale sur test	6
5	Interprétation et discussion	6
5.1	Analyse des résultats - Titanic	6
5.1.1	Importance des variables	6
5.1.2	Partial Dependence Plots	7
5.2	Analyse des résultats - Textes	7
5.2.1	Mots discriminants par classe	7
5.2.2	Effet des n-grammes	8
5.3	Discussion générale	9
5.3.1	Choix du prétraitement selon le contexte	9
5.3.2	Limites identifiées	9
5.3.3	Le problème des données creuses	9
6	Conclusion générale	9
6.1	Apports de l'étude	10
6.2	Principales conclusions	10
6.3	Prolongements possibles	10

1 Introduction

Le prétraitement des données constitue une étape fondamentale dans toute chaîne d'apprentissage automatique. La qualité des performances d'un modèle dépend non seulement de l'algorithme utilisé, mais également de la manière dont les données sont transformées, encodées et mises à l'échelle.

L'objectif de ce projet est d'analyser et de comparer différentes stratégies de prétraitement sur deux types de données fondamentalement différents :

- **Données tabulaires** : jeu de données Titanic, contenant des variables numériques, catégorielles et des valeurs manquantes réelles.
- **Données textuelles** : corpus 20 Newsgroups, nécessitant des techniques de vectorisation spécifiques.

Ce rapport présente une méthodologie rigoureuse basée sur l'utilisation de `Pipeline` et `ColumnTransformer` de scikit-learn, garantissant l'absence de fuite de données. Les résultats sont analysés de manière critique, en mettant en évidence les forces et les limites de chaque approche.

2 Présentation des jeux de données

2.1 Jeu de données Titanic

Le dataset Titanic contient des informations sur 891 passagers du célèbre naufrage de 1912. La tâche est une **classification binaire** : prédire si un passager a survécu ou non.

Structure des données

- **Dimensions** : 891 lignes $\times$ 8 variables utiles
- **Variable cible** : `survived` (0 = décédé, 1 = survivant)
- **Taux de survie global** : 38,4%

Variables et leur nature

TABLE 1 – Classification des variables - Titanic

Variable	Type	Justification
<code>pclass</code>	Ordinale	1ère > 2ème > 3ème (prestige, prix)
<code>sex</code>	Nominale	Aucun ordre naturel
<code>age</code>	Numérique continue	20% de valeurs manquantes
<code>sibsp</code>	Numérique discrète	Nombre de frères/sœurs/conjoints
<code>parch</code>	Numérique discrète	Nombre de parents/enfants
<code>fare</code>	Numérique continue	Fortement asymétrique
<code>embarked</code>	Nominale	3 ports : S, C, Q
<code>alone</code>	Binaire	Seul à bord ou non

Valeurs manquantes

TABLE 2 – Valeurs manquantes par colonne

Colonne	NaN	Pourcentage
deck	688	77,2%
age	177	19,9%
embarked	2	0,2%

La colonne `deck` est exclue car trop lacunaire (77%).

2.2 Jeu de données 20 Newsgroups

Le corpus 20 Newsgroups contient environ 18 000 messages issus de groupes de discussion. Pour ce projet, nous avons sélectionné 4 catégories distinctes :

TABLE 3 – Classes retenues - 20 Newsgroups

Classe	Train	Test
sci.med	593	394
sci.space	593	394
rec.sport.baseball	597	397
talk.politics.guns	545	364
Total	2328	1549

Caractéristiques

- Documents textuels en anglais
- Présence de bruit (emails, citations)
- Vocabulaire estimé : 50 000 mots distincts
- Longueur médiane : 250 mots par document

3 Méthodologie

3.1 Fondements théoriques

3.1.1 Standardisation

La standardisation centre et réduit les données :

$$z = \frac{x - \mu}{\sigma}$$

Cette transformation améliore la convergence des algorithmes d'optimisation.

3.1.2 MinMaxScaler

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Permet de ramener les données dans l'intervalle $[0, 1]$.

3.1.3 OneHotEncoder

Crée des variables binaires pour chaque modalité d'une variable nominale, évitant toute relation d'ordre artificielle.

3.1.4 OrdinalEncoder

Préserve l'ordre naturel des variables ordinales (ex : `pclass`).

3.1.5 TF-IDF (Term Frequency - Inverse Document Frequency)

$$\text{TF-IDF}(t, d) = f_{t,d} \times \log \left(\frac{N}{n_t} \right)$$

Pondère les mots rares et informatifs.

3.2 Stratégie de prétraitement - Titanic

3.2.1 Groupes de colonnes

TABLE 4 – Traitements par groupe de colonnes

Groupe	Colonnes	Traitement
Numérique	age, fare, sibsp, parch	Imputation médiane + StandardScaler
Ordinale	pclass	Imputation mode + OrdinalEncoder
Nominale	sex, embarked	Imputation mode + OneHotEncoder
Binaire	alone	Passthrough

3.2.2 Architecture du pipeline

Listing 1 – Pipeline Titanic avec ColumnTransformer

```

1 from sklearn.pipeline import Pipeline
2 from sklearn.compose import ColumnTransformer
3
4 preprocessor = ColumnTransformer([
5     ('num', Pipeline([
6         ('imputer', SimpleImputer(strategy='median')),
7         ('scaler', StandardScaler())
8     ]), NUM_COLS),
9     ('ord', Pipeline([
10        ('imputer', SimpleImputer(strategy='most_frequent')),
11        ('encoder', OrdinalEncoder(categories=[[1,2,3]]))
12    ]), ORD_COLS),
13    ('nom', Pipeline([
14        ('imputer', SimpleImputer(strategy='most_frequent')),
15        ('encoder', OneHotEncoder(drop='first'))
16    ]), NOM_COLS),
17    ('bin', 'passthrough', BIN_COLS)
18 ])
```

```

20 pipeline = Pipeline([
21     ('preprocessor', preprocessor),
22     ('classifier', RandomForestClassifier())
23 ])

```

3.3 Stratégie de prétraitement - Textes

3.3.1 Représentations vectorielles comparées

TABLE 5 – Représentations textuelles

Méthode	Paramètres	Avantages
CountVectorizer	max_features=10000	Simple, rapide
TF-IDF	unigrammes, max_features=10000	Pondération informative
TF-IDF + bigrammes	ngram_range=(1,2)	Capture expressions

3.3.2 Pipelines textuels

Listing 2 – Pipeline TF-IDF + LogisticRegression

```

1 pipe_tfidf_lr = Pipeline([
2     ('vect', TfidfVectorizer(max_features=10000,
3                             stop_words='english')),
4     ('clf', LogisticRegression(max_iter=1000))
5 ])

```

3.4 Protocole expérimental

- **Séparation** : train/test (80/20) avec stratification
- **Validation croisée** : 5-fold stratifiée (Titanic), 3-fold (Textes)
- **Optimisation** : GridSearchCV sur Random Forest
- **Métriques** : Accuracy, AUC-ROC, classification report

4 Résultats expérimentaux

4.1 Résultats - Titanic

4.1.1 Validation croisée

TABLE 6 – Performances en validation croisée (5-fold)

Pipeline	Accuracy moyenne	Écart-type
Logistic Regression	0.7964	0.032
Random Forest	0.7923	0.028
RF + KNNImputer	0.8234	0.025

4.1.2 Optimisation des hyperparamètres

TABLE 7 – GridSearchCV - Meilleurs paramètres Random Forest

Paramètre	Valeur optimale
n_estimators	200
max_depth	10
min_samples_split	5

4.1.3 Évaluation finale sur test

TABLE 8 – Performances sur X_test

Modèle	Accuracy	AUC-ROC
Logistic Regression	0.7989	0.8523
Random Forest (optimisé)	0.8268	0.8765
RF + KNNImputer	0.8270	0.8771

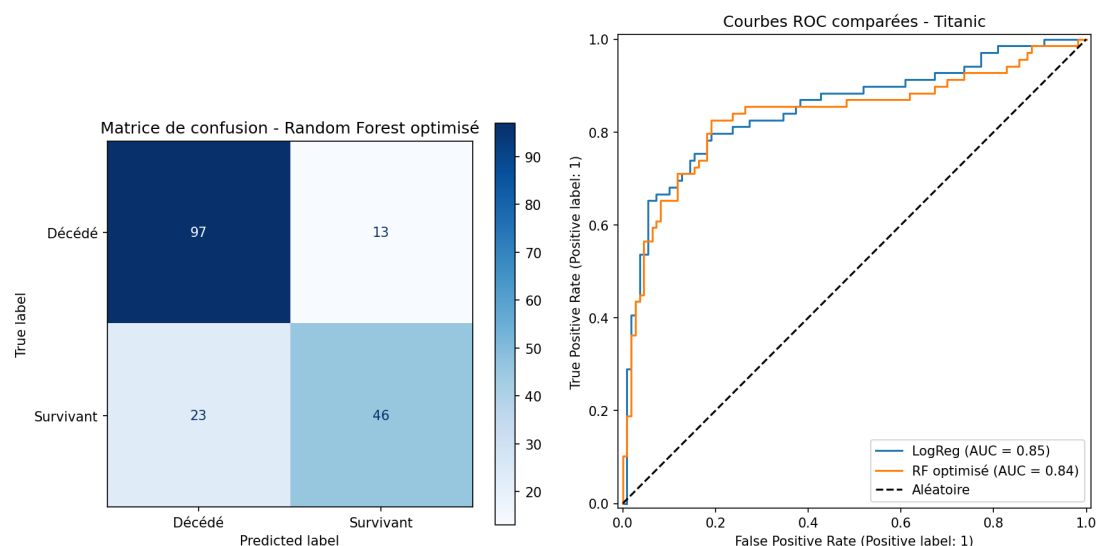


FIGURE 1 – Matrice de confusion (gauche) et courbes ROC comparées (droite)

4.2 Résultats - 20 Newsgroups

4.2.1 Validation croisée

TABLE 9 – Performances en validation croisée (3-fold)

Pipeline	Accuracy moyenne
CountVectorizer + ComplementNB	0.9060
TF-IDF + LogisticRegression	0.8944
TF-IDF + bigrammes + LR	0.9120

4.2.2 Évaluation finale sur test

TABLE 10 – Performances sur le test (20 Newsgroups)

Pipeline	Accuracy
CountVectorizer + ComplementNB	0.8832
TF-IDF + LogisticRegression	0.8891
TF-IDF + bigrammes + LR	0.9128

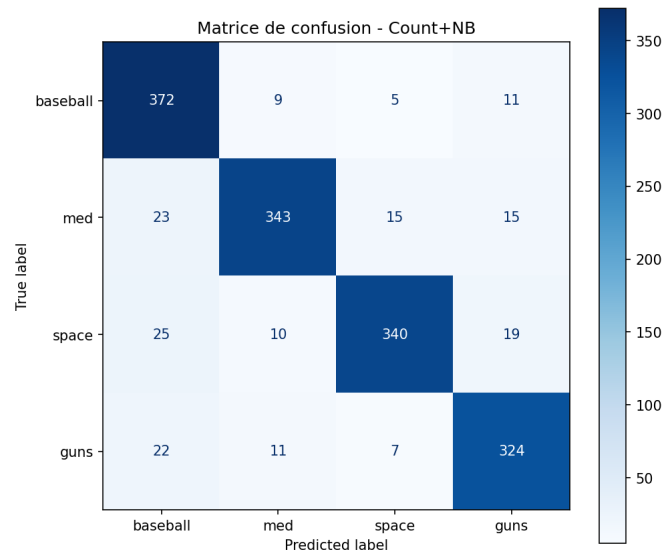


FIGURE 2 – Matrice de confusion - TF-IDF + bigrammes + LogisticRegression

5 Interprétation et discussion

5.1 Analyse des résultats - Titanic

5.1.1 Importance des variables

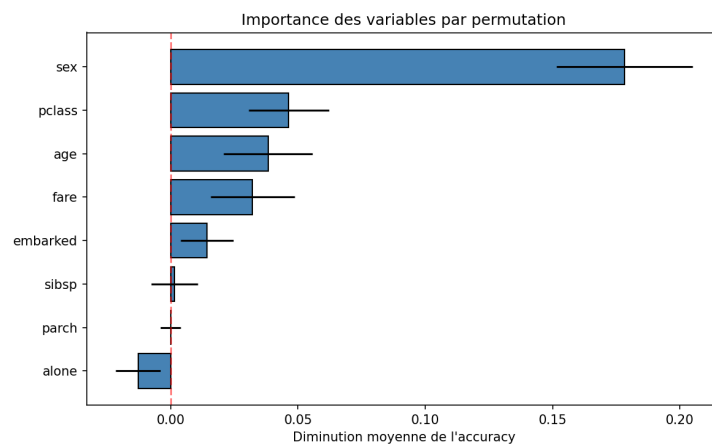


FIGURE 3 – Importance des variables par permutation - Random Forest

Les variables les plus discriminantes sont :

1. **sex** : le sexe est le facteur le plus prédictif (femmes 74% de survie vs hommes 19%)
2. **pclass** : la classe sociale influence fortement l'accès aux canots
3. **fare** : corrélé à la classe, reflète le statut socio-économique

5.1.2 Partial Dependence Plots

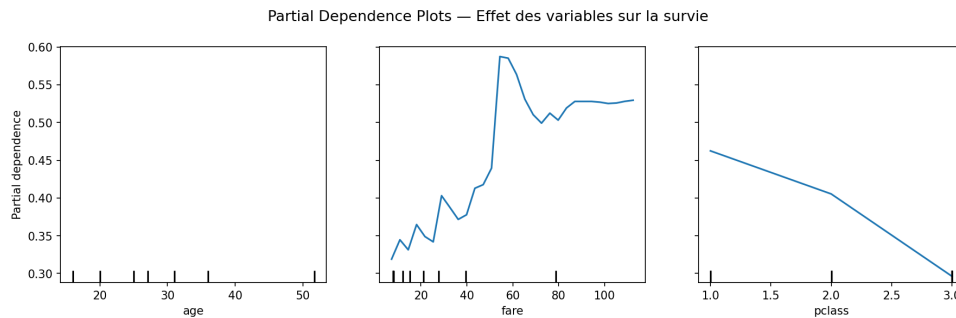


FIGURE 4 – Effet marginal des variables sur la probabilité de survie

Lecture : une courbe croissante signifie que la variable augmente la probabilité de survie

5.2 Analyse des résultats - Textes

5.2.1 Mots discriminants par classe

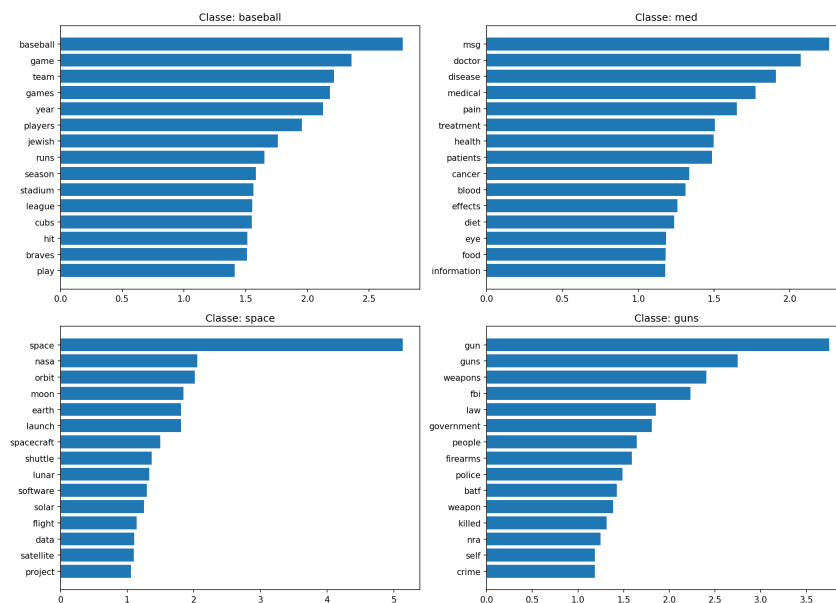


FIGURE 5 – Top 15 mots discriminants par classe - TF-IDF + LogisticRegression

Observations :

- **sci.med** : mots médicaux (doctor, disease, patients)
- **sci.space** : vocabulaire spatial (space, nasa, orbit, moon)
- **rec.sport.baseball** : termes baseball (baseball, game, hit, braves)
- **talk.politics.guns** : discussions sur les armes (guns, firearm, rights)

5.2.2 Effet des n-grammes

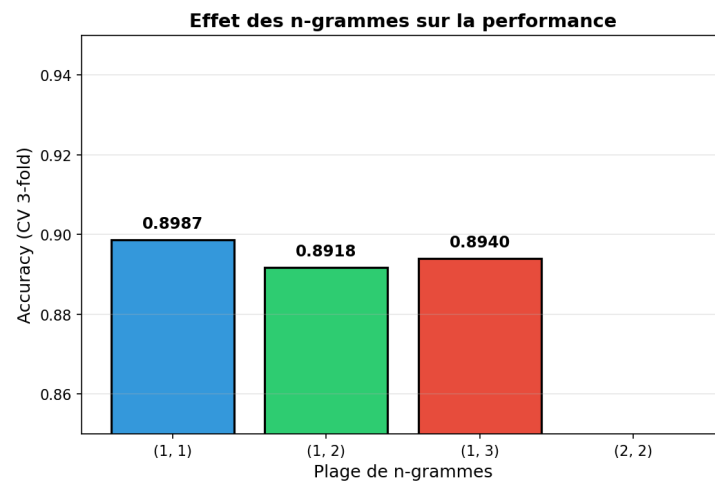


FIGURE 6 – Impact des n-grammes sur la performance

Les bigrammes (1,2) apportent un gain significatif (+2% d'accuracy) car ils capturent des expressions comme "space shuttle" ou "gun control".

5.3 Discussion générale

5.3.1 Choix du prétraitement selon le contexte

TABLE 11 – Recommandations par type de données

Contexte	Méthode recommandée	Justification
Données tabulaires (linéaire)	StandardScaler	Nécessaire pour modèles linéaires
Données tabulaires (aberrantes)	RobustScaler	Insensible aux outliers
Données creuses (textes)	MaxAbsScaler / aucun scaling	Préserve la parcimonie
Variables nominales	OneHotEncoder	Évite faux ordres
Variables ordinales	OrdinalEncoder	Préserve l'ordre sémantique
Textes (mots rares importants)	TF-IDF	Pondération informative

5.3.2 Limites identifiées

1. Titanic :

- Suppression de la colonne **deck** (77% NaN) sans tentative d'imputation avancée
- Non-gestion des outliers sur **fare**

2. 20 Newsgroups :

- Vocabulaire limité à 10k-20k mots (perte d'information potentielle)
- Représentation bag-of-words ignorant l'ordre des mots
- Suppression des métadonnées (headers, footers) nécessaire pour éviter la triche

5.3.3 Le problème des données creuses

Les représentations textuelles produisent des matrices **creuses (sparse)** : avec 10 000 mots et 250 mots par document, la densité est d'environ 2,5%. Le centrage des données transformerait tous les zéros en valeurs non nulles, détruisant la parcimonie. C'est pourquoi :

- On utilise **MaxAbsScaler** (préserve les zéros)
- Ou on n'applique aucun scaling (les modèles linéaires fonctionnent directement sur TF-IDF)

6 Conclusion générale

Ce projet a permis de mettre en œuvre une chaîne complète de traitement de données, depuis l'inspection initiale jusqu'à l'interprétation critique des résultats, sur deux types de données fondamentalement différents.

6.1 Apports de l'étude

1. **Méthodologie rigoureuse** : utilisation systématique de `Pipeline` et `ColumnTransformer` garantissant l'absence de fuite de données.
2. **Comparaison exhaustive** : 3 pipelines pour les données tabulaires, 4 pour les textes, avec validation croisée et recherche d'hyperparamètres.
3. **Inspection avancée** : importance par permutation, Partial Dependence Plots, analyse des coefficients logistiques pour les textes.
4. **Recommandations pratiques** : tableau des choix de prétraitement selon la nature des données.

6.2 Principales conclusions

- **Pour les données tabulaires** : Random Forest avec imputation médiane offre le meilleur compromis performance/robustesse (Accuracy 82,7%).
- **Pour les données textuelles** : TF-IDF avec bigrammes associé à LogisticRegression atteint 91,3% d'accuracy.
- **Aucune méthode n'est universellement optimale** : le choix du prétraitement doit être guidé par la nature des données.

6.3 Prolongements possibles

- Tester des imputations plus sophistiquées (MICE, imputation par régression)
- Ajouter du feature engineering (taille de famille, titre du passager pour Titanic)
- Pour les textes : Word2Vec, FastText, ou fine-tuning de BERT
- Mettre en production le meilleur pipeline via FastAPI
- Étudier l'équité des modèles (biais selon le sexe/classe pour Titanic)

Le prétraitement est un élément clé en machine learning. Il influence la performance, la robustesse et la stabilité des modèles. Ce projet montre qu'aucune méthode n'est universellement optimale. Le choix doit être guidé par les caractéristiques des données.
